AZURE COST VISIBILITY AND GUARDRAILS

A cost feedback loop for Azure, built with Terraform | Version 1

Tiered and forecast budget alerts, anomaly detection, a weekly cost report, and a cost-attribution policy

Project	Azure Cost Visibility and Guardrails
Version / date	Version 1, October 27, 2026
Author	Bryan Sykes
Platform	Microsoft Azure (Cost Management, Monitor, Policy)
IaC tool	Terraform 1.16.4, provider hashicorp/azurerm v3.117.1
Remote state	Azure Blob (TerraformStateRG / sykestfstate), own key: cost-governance.terraform.tfstate
Companion project	Personal Azure Infrastructure: IaC via Terraform (hub-and-spoke network)
GitHub	github.com/MrBSykes/azure-cost-visibility-guardrails

1. The Business Problem

> **Fictional scenario, built to practice a real-world problem**
> A non-profit moved its operations into Azure and, for the first few months, stayed within budget and paid on time. As the team deployed more resources, monthly spend climbed. Nobody was tracking it, and nobody owned it. By month six, leadership was cutting budgets in other programs to cover a growing cloud bill. Going into month seven, the organization nearly missed a payment, and the manager learned about the overrun from the invoice, not from any warning.

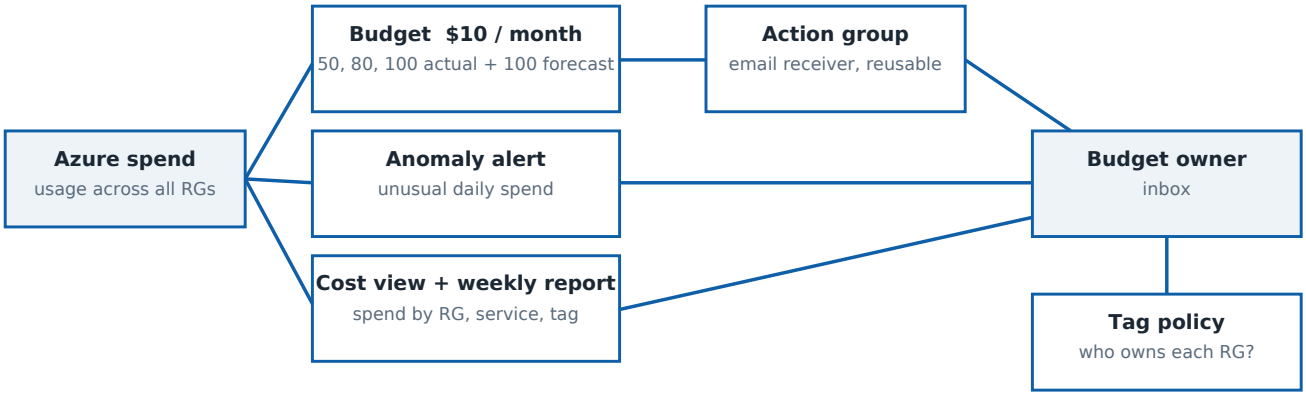
The root problem is the absence of a feedback loop. Cost was visible only after it was incurred, and no one could say which resources or projects were driving it. The organization did not overspend because it was careless. It overspent because nothing told it what was happening while there was still time to act.

Failure	What it caused	What this project adds
No early warning	Manager found out from the invoice	Tiered budget alerts, including a forecast alert that fires before the limit is hit
Report nobody reads	Spend visible only to people who open the portal	A weekly cost report emailed to the manager
Surprises between checks	A forgotten resource ran for weeks	Anomaly alert on unusual daily spend
No ownership	Could not say which project caused a spike	A tag policy that reports resource groups with no cost-center tag

Intended outcome: the budget owner hears about a spending problem within about a day, before the month ends, and can say which project caused it. Azure cost data lags by hours, so this is near-real-time, not instant.

2. How the Feedback Loop Works

All of the governance resources live in their own resource group and their own Terraform state file, so they are independent of the workloads they watch. The alerts and the report all lead to the same place: the person who owns the budget.



Resource	Terraform type	Purpose
monthly-subscription-budget	azurerm_consumption_budget_subscription	$10 per month with four alert conditions
cost-alerts-ag	azurerm_monitor_action_group	Reusable email notification target
spend-by-resource-group	azurerm_subscription_cost_management_view	Saved Cost Analysis view
weekly-cost-report	azurerm_cost_management_scheduled_action	Emails that view every Monday
daily-anomaly-alert	azurerm_cost_anomaly_alert	Flags unusual daily spend
Cost attribution tag policy	azurerm_policy_definition + subscription_policy_assignment	Reports resource groups missing the cost-center tag
cost-governance-rg	azurerm_resource_group	Holds the action group

3. Component Deep Dives

Each component answers four questions: what it does in Azure, what business problem it solves, how it was built, and what I would do differently.

3.1 Budget with tiered and forecast alerts

What it does in Azure	A budget tracks spend against an amount for a period and sends notifications when thresholds are crossed. Notifications can be based on actual spend or on forecasted spend, which Azure projects from usage so far.
Business problem	Actual-only alerts arrive after the money is spent. A forecast alert predicts the month-end total while there is still time to act, which is exactly what the non-profit lacked.
How it was built	A $10 monthly budget at subscription scope with four notifications: 50% actual (awareness), 80% actual (planning), 100% forecasted (early warning), 100% actual (action). All four route to the action group, which is also under the five-notification limit per budget.
Next time	Create the budget on day one of the subscription, before the first deployment. Add per-team budgets filtered by tag, and add an automated response to the 100% alert, because alerts inform but do not stop spending.

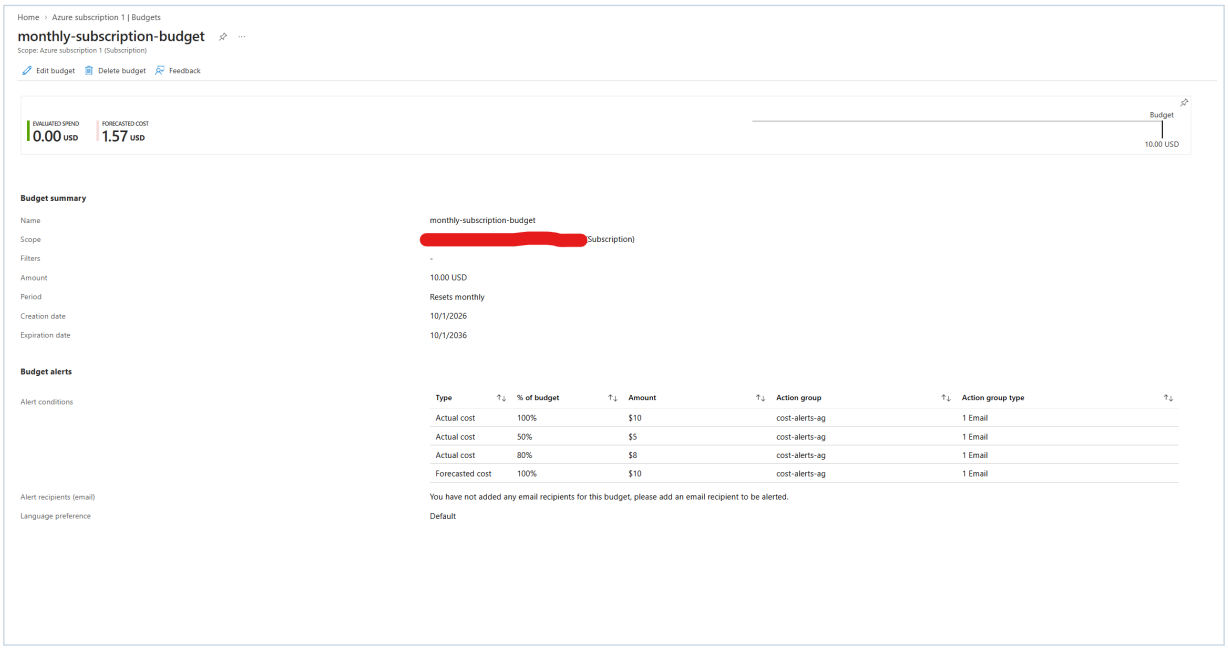
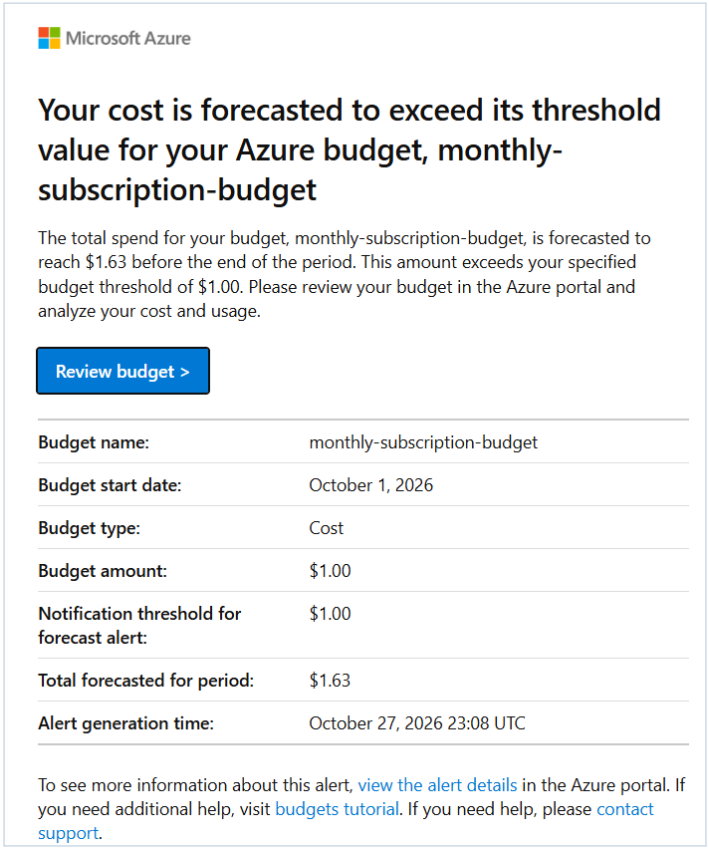


Figure 1. Budget with four alert conditions routed to cost-alerts-ag. The "no email recipients" note refers to direct emails; alerts go through the action group. (screenshots/01-budget-alert-thresholds.png)



Microsoft Azure

Your cost is forecasted to exceed its threshold value for your Azure budget, monthly-subscription-budget

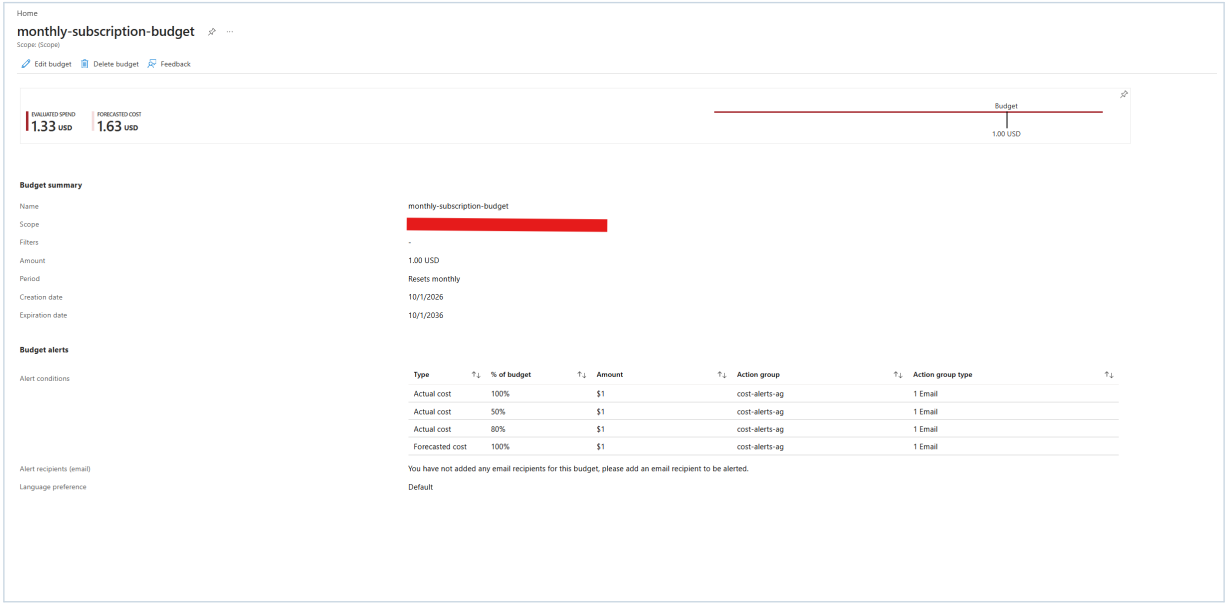
The total spend for your budget, monthly-subscription-budget, is forecasted to reach $1.63 before the end of the period. This amount exceeds your specified budget threshold of $1.00. Please review your budget in the Azure portal and analyze your cost and usage.

Review budget >

Budget name:	monthly-subscription-budget
Budget start date:	October 1, 2026
Budget type:	Cost
Budget amount:	$1.00
Notification threshold for forecast alert:	$1.00
Total forecasted for period:	$1.63
Alert generation time:	October 27, 2026 23:08 UTC

To see more information about this alert, view the alert details in the Azure portal. If you need additional help, visit budgets tutorial. If you need help, please contact support.

Figure 2. A real forecast alert. With the budget temporarily lowered to $1, Azure emailed that month-end spend was forecast at $1.63, over the $1.00 threshold. (screenshots/06-forecast-alert-email.png)



Home

monthly-subscription-budget

Scope: (Scope)

Edit budget | Delete budget | Feedback

EVALUATED SPEND 1.33 USD | FORECASTED COST 1.63 USD

Budget 1.00 USD

Budget summary

Name	monthly-subscription-budget
Scope	[redacted]
Filters	-
Amount	1.00 USD
Period	Resets monthly
Creation date	10/1/2026
Expiration date	10/1/2036

Budget alerts

Alert conditions

Type	% of budget	Amount	Action group	Action group type
Actual cost	100%	$1	cost-alerts-ag	1 Email
Actual cost	50%	$1	cost-alerts-ag	1 Email
Actual cost	80%	$1	cost-alerts-ag	1 Email
Forecasted cost	100%	$1	cost-alerts-ag	1 Email

Alert recipients (email): You have not added any email recipients for this budget, please add an email recipient to be alerted.

Language preference: Default

Figure 3. The same budget during the test, set to $1: evaluated spend of $1.33 is over the limit, with a $1.63 forecast. (screenshots/07-budget-exceeded-test.png)

3.2 Action group

What it does in Azure	An action group is a reusable set of notification targets. Budgets and alert rules point at it instead of listing emails themselves.
Business problem	When the budget owner changes, one place changes, not every alert. It also lets SMS, a webhook, or an automation runbook be added later without touching the budget.
How it was built	One email receiver with the address supplied through a variable (TF_VAR_alert_email), so the real address never reaches Git. Azure sends a verification email when a receiver is added; receiving it confirmed the path to the inbox works.
Next time	Add a second receiver so the alert has no single point of failure, and a runbook action for automated cost response.

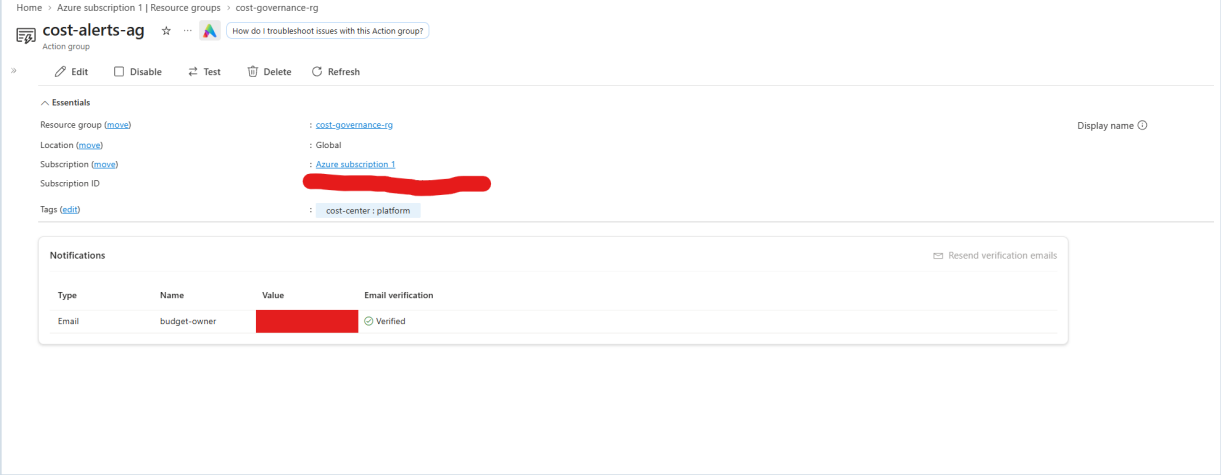


Figure 4. cost-alerts-ag with the email receiver Verified and the cost-center tag applied. (screenshots/02-action-group-verified.png)

3.3 Cost view and weekly report

What it does in Azure	A saved Cost Analysis view stores a chart configuration. A scheduled action emails that view on a recurring schedule.
Business problem	A non-technical manager will not open the portal to check spend. Pushing a short report to their inbox every Monday turns cost visibility from something they must remember into something that arrives.
How it was built	A month-to-date view grouped by resource group, plus pivots for service and the cost-center tag, emailed weekly on Mondays at 13:00 UTC. I chose an emailed view over a hand-built dashboard because dashboard JSON is brittle and a pushed report gets read.
Next time	Add a monthly report by tag for project owners, and consider exporting cost data to storage for longer-term analysis.

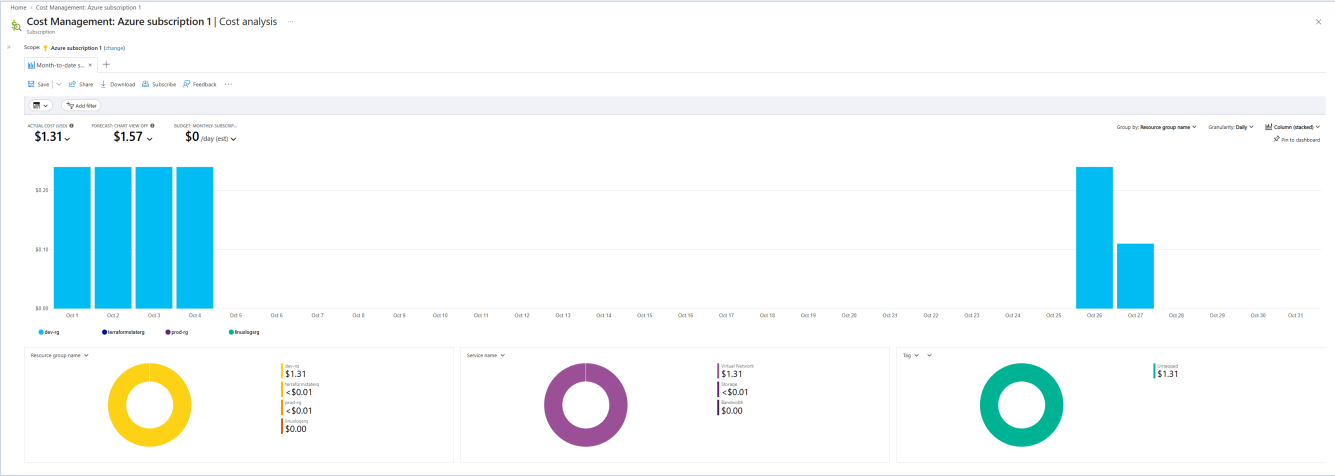


Figure 5. Cost analysis from the saved view: $1.31 month-to-date, $1.57 forecast, almost entirely dev-rg and billed under Virtual Network, all Untagged. (screenshots/03-cost-analysis-by-resource-group.png)

3.4 Anomaly alert

What it does in Azure	Cost Management compares daily spend with expected patterns and emails when something looks unusual.
Business problem	A budget only catches a problem when the total crosses a line. An anomaly alert catches a sudden jump, such as a forgotten resource or a misconfigured service, early in the month.
How it was built	A subscription-scoped daily anomaly alert emailing the budget owner. Per the provider documentation, anomaly emails are sent based on the access of the person who created the rule, so that person must keep their access.
Next time	Scope an alert per resource group for the highest-risk environments, and send it to the action group so every alert follows one path.

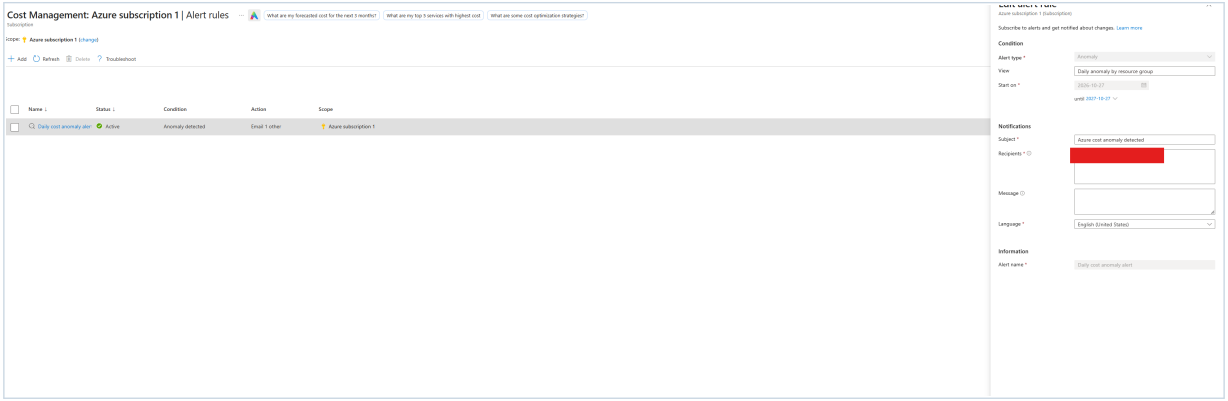


Figure 6. Anomaly alert rule: active, subscription scope, daily. (screenshots/04-anomaly-alert-rule.png)

3.5 Cost-attribution tag policy

What it does in Azure	Azure Policy evaluates resources against rules. A policy can audit (report non-compliance) or deny (block creation). Policy mode matters: Indexed mode skips resource groups, so this rule uses mode All.
Business problem	Alerts say that spend is high. Tags say whose spend it is. Without an owner tag, a spike cannot be traced to a project or a team.
How it was built	A custom definition requiring a cost-center tag on resource groups, with parameters for the tag name and the effect, assigned to the subscription. It starts in Audit mode, because existing untagged groups (including my network project) would block updates under Deny.
Next time	Tag every resource group at creation, then switch to Deny. Add the built-in policy that inherits a tag from the resource group so resources carry it too.

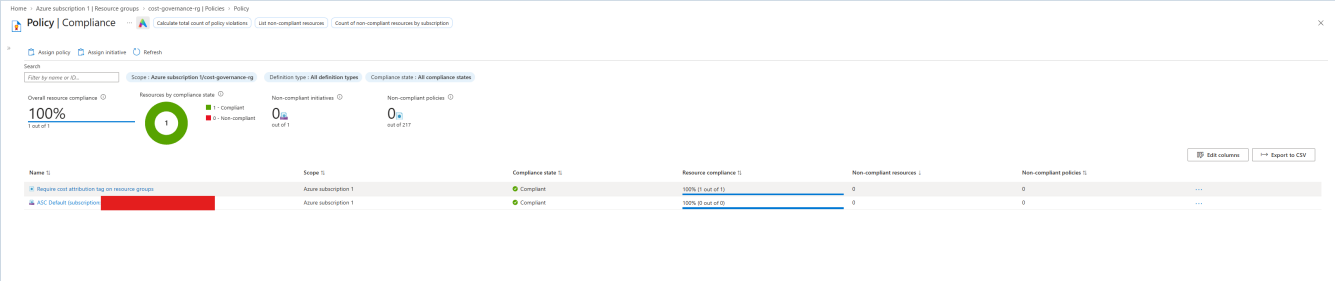


Figure 7. Policy compliance viewed at the cost-governance-rg scope (1 of 1 compliant). A subscription-wide view of untagged groups is a planned follow-up. (screenshots/05-policy-compliance.png)

4. Findings From My Own Subscription

Running the loop against a real subscription produced a real finding, which is the point of the project:

- **The spend is concentrated.** Month-to-date spend was $1.31 with a $1.57 forecast, and nearly all of it sat in dev-rg under Virtual Network. That is consistent with the dev storage private endpoint, which bills hourly. The two storage accounts and the state account cost under a cent.
- **None of it was attributed.** The tag breakdown showed the full amount as Untagged. A manager looking at this report could see where the money went but not whose project it belonged to, which is the gap the tag policy reports.
- **The alert path works end to end.** After I lowered the budget to $1 as a test, Azure emailed a forecast alert (month-end spend forecast at $1.63 against a $1.00 threshold) after the change, proving the chain from budget to action group to inbox.
- **Budgets lag spend.** The budget page showed $0.00 evaluated spend while cost analysis showed $1.31. Azure evaluates budgets on a schedule and cost data arrives hours late, so alerts are not instant. The documentation says "within about a day" for this reason.

5. Resources Deployed

Eight Azure resources are deployed through Terraform in one apply sequence, with state in a separate key from the network project.

#	Resource	Name	Status
1	Resource group	cost-governance-rg (East US)	Deployed
2	Action group	cost-alerts-ag	Deployed
3	Budget	monthly-subscription-budget ($10)	Deployed

#	Resource	Name	Status
4	Cost Management view	spend-by-resource-group	Deployed
5	Scheduled action	weekly-cost-report	Deployed (portal screenshot pending)
6	Anomaly alert	daily-anomaly-alert	Deployed
7	Policy definition	require-cost-tag-on-resource-groups	Deployed
8	Policy assignment	require-cost-tag-on-rg (Audit)	Deployed

6. Troubleshooting Log

#	Symptom	Root cause	Resolution	Lesson
1	Cost view failed: 400 "Non table views should have three pivots"	Cost Analysis charts other than tables require exactly three pivots. The provider docs say so and I missed it	Added three pivot blocks: resource group, service, and the cost-center tag	Read the argument reference for required blocks, and expect the service API to enforce rules the schema does not
2	The same error after the fix	The edited file had not been saved to disk, so Terraform ran the old code. A plan showing "2 to add" looked identical either way	Verified the file contents with grep before re-running; apply then succeeded	Confirm the code on disk matches what you believe you are applying
3	Budget showed $0.00 spend while cost analysis showed $1.31	Budgets are evaluated on a schedule, and cost data lags by hours	Documented the delay and worded the outcome as "within about a day"	Never promise real-time on a system with a data pipeline delay
4	Compliance view showed 1 of 1 compliant, not the untagged groups	The view was scoped to cost-governance-rg, which carries the tag	Captioned the screenshot honestly; a subscription-scope view is a follow-up	Check the scope before reading a compliance result

7. Cost and Limitations

Budgets, action groups (email), cost views, scheduled reports, anomaly alerts, and policy assignments carry no charge as far as I know, and I have not seen a charge for them in this subscription. Nothing here deploys billable compute or networking. The claims I have not verified are marked as such in the README.

Honest limits of this design:

- **Budgets alert; they do not stop spending.** An automated response, such as a runbook that stops non-essential resources at 100%, is the natural next step.
- **Alerts are not instant.** Cost data lags by hours and budgets are evaluated on a schedule.
- **One subscription.** A larger organization would scope budgets at a management group and add per-team budgets filtered by tag.
- **Anomaly alerts depend on the creator keeping access.** A real deployment should name a role-based owner, not a person.
- **The policy audits, it does not enforce.** It reports untagged groups until they are fixed, then it can be switched to Deny.

8. Business and Career Alignment

Cost governance is a core cloud engineering responsibility, often called FinOps. This project shows the practical habits behind it:

Skill	Evidence in this project
Infrastructure as Code	Eight resources defined in Terraform, with parameters, validation, and a separate state key
Cost management (FinOps)	Tiered and forecast budgets, anomaly detection, scheduled reporting, and attribution by tag
Governance	A custom Azure Policy definition and assignment, with a deliberate Audit-before-Deny rollout
Monitoring and alerting	Action group design that separates who is notified from what triggers the alert
Secure engineering habits	Email and subscription identifiers passed through environment variables and kept out of Git; screenshots redacted
Verification and honesty	Found a deployment error the plan did not reveal, documented the real limits, and marked unverified claims

This supports the governance and cost-management objectives of the Azure Administrator (AZ-104) path, which is my next certification goal, alongside AZ-900 and CompTIA Security+ already completed.

9. What I Would Do Differently Next Time

- Create the budget and alerts on day one of the subscription, before the first deployment.
- Decide the tag standard up front and tag every resource group at creation.
- Scope budgets per team or project, not only per subscription.
- Add an automated response to the 100% alert, not just an email.
- Send all alerts through the action group so there is one notification path.
- Verify that the code on disk matches the plan before applying, especially after an edit that did not seem to take.

Next steps

Action	Purpose
Add the weekly-report portal screenshot	Complete the evidence set
Tag existing resource groups, then switch the policy to Deny	Move from reporting to enforcement
Per-tag budget for one project	Show per-team accountability
Automated response to the 100% alert	Turn a notification into a guardrail
CI that runs fmt, validate, and plan	Review every change before it deploys

10. Conclusion

This project closes the gap that hurt the fictional non-profit: not a lack of discipline, but a lack of feedback. In about eight resources it gives a budget owner early warning, a regular report, a way to spot unusual spend, and a way to say whose spend it is, and it is defined entirely in code that can be reviewed, repeated, and torn down.

Running it against my own subscription also did what a cost loop is supposed to do: it showed that nearly all of my spend came from one resource group, none of it attributed to an owner. The problems along the way, a missing required block and an edit that did not save, are the everyday work of infrastructure engineering.

Documentation prepared by Bryan Sykes, October 27, 2026.